Dijkstra (Shortest Distance) vs. BFS (Fewest Highway Switches) in Real Road Networks

CSIS 385: Design and Analysis of Algorithms

Siena University · Spring 2026

Team Members: Dilan, Sam Panackal, Samiullah Neyazy
Repository: *(https://github.com/samineyazy25/AcademicShowcase.git)*
Submission Date: May 4, 2026

---

1. Project Overview

This project investigates two fundamentally different approaches to finding optimal routes in highway networks. We compared **Dijkstra's Algorithm**, which computes the shortest path by total distance, against a **modified Breadth-First Search (BFS)** on a layered graph, designed to minimize the number of highway switches a driver must make along a route.

While shortest-distance routing is the standard approach used in most GPS systems, drivers often prefer routes that stay on the same highway for as long as possible — even if the total distance is slightly longer. Fewer highway transitions means fewer decisions, fewer exits to watch for, and a simpler driving experience overall. This project explores how frequently these two objectives agree, and what tradeoffs exist when they diverge.

Our road network data was sourced from **METAL (Map-based Educational Tools for Algorithm Learning)**, which provides graph data derived from the Travel Mapping project. Vertices represent highway intersections and edges represent road segments tagged with both a distance and a highway identifier.

---

2. Algorithms Studied

2.1 Dijkstra's Algorithm

Dijkstra's algorithm finds the single-source shortest path in a weighted graph with non-negative edge weights. Our implementation uses a **priority queue** variant for the TMG (METAL) graph

data, where edges are stored as an adjacency list and distances are computed as real-valued miles.

How it works:

```
 public static double dijkstraTMG(int start, int end) {
        double[] dist = new double[n];
        Arrays.fill(dist, Double.MAX_VALUE);
        dist[start] = 0.0;
        PriorityQueue<double[]> pq = new PriorityQueue<>(Comparator.comparingDouble(a ->
a[0]));
        pq.offer(new double[]{0.0, start});
        while (!pq.isEmpty()) {
            double[] curr = pq.poll();
            int u = (int) curr[1];
            if (curr[0] > dist[u]) continue;
            if (u == end) return dist[end];
            for (int[] edge : adjList[u]) {
                int v = edge[0];
                double newDist = dist[u] + edge[1];
                if (newDist < dist[v]) {
                    dist[v] = newDist;
                    pq.offer(new double[]{newDist, v});
                }
            }
        }
        return dist[end] == Double.MAX_VALUE ? -1 : dist[end];
    }
```

Each entry in the priority queue is a `double[]` pair of `{distance, vertex}`. The line `if (curr[0] > dist[u]) continue` is a lazy deletion check. Since `PriorityQueue` does not support decrease-key, we allow stale entries and skip them when popped. Edge weights `edge[1]` are stored as integer millimiles (distance × 1000) during TMG loading, so the returned value is divided by 1000 to get miles in the main method.

Time complexity: O((V + E) log V) with a priority queue on adjacency list graphs; $O(V^2)$ for our synthetic matrix-based testing version.

What it optimizes: Total path distance in miles. Produces the provably shortest route between two points.

2.2 Modified BFS — Minimizing Highway Switches

Our modified BFS is built on a **0-1 weighted deque (double-ended queue)** applied to a **layered graph** where each state is a `(vertex, highway)` pair. Traversing an edge on the same highway costs 0 switches; switching to a different highway costs 1 switch. This is a classic 0-1 BFS pattern, where zero-cost edges push to the front of the deque and one-cost edges push to the back.

How it works:

```
public static int bfsTMG(int start, int end) {
    int H = nextHighwayId;
    int[][] minSwitches = new int[n][H];
    for (int[] row : minSwitches) Arrays.fill(row, Integer.MAX_VALUE);
    Deque<int[]> dq = new ArrayDeque<>();
    for (int[] edge : adjList[start]) {
        int hId = edge[2];
        if (minSwitches[start][hId] > 0) {
            minSwitches[start][hId] = 0;
            dq.addFirst(new int[]{start, hId, 0});
        }
    }
    while (!dq.isEmpty()) {
        int[] s = dq.pollFirst();
        int vertex = s[0], hId = s[1], switches = s[2];
        if (switches > minSwitches[vertex][hId]) continue;
        if (vertex == end) break;
        for (int[] edge : adjList[vertex]) {
            int nextV = edge[0], nextH = edge[2];
            boolean sameHW = (nextH == hId);
            int newSwitches = switches + (sameHW ? 0 : 1);
            if (newSwitches < minSwitches[nextV][nextH]) {
                minSwitches[nextV][nextH] = newSwitches;
                if (sameHW) dq.addFirst(new int[]{nextV, nextH, newSwitches});
                else dq.addLast(new int[]{nextV, nextH, newSwitches});
            }
        }
    }
    int best = Integer.MAX_VALUE;
    for (int h = 0; h < H; h++) {
        if (minSwitches[end][h] < best) best = minSwitches[end][h];
    }
    return best == Integer.MAX_VALUE ? -1 : best;
}
```

- State space: `minSwitches[v][h]` tracks the minimum switches to reach vertex `v` while currently on highway `h`. This is the layered graph, meaning each (vertex, highway) pair is a distinct state.
- 0-1 deque: `dq.addFirst` for same-highway edges (0 cost) and `dq.addLast` for highway switches (1 cost). This guarantees states are processed in non-decreasing switch order without a full sort, equivalent to Dijkstra on a 0-1 weighted graph.
- Stale check: `if (switches > minSwitches[vertex][hId]) continue` discards outdated states, same as Dijkstra's deletion.

Best path recovery: After reaching the end, we scan all highways to find which arrival highway produced the minimum switch count.

Time complexity: O(V × H) where H is the number of distinct highway identifiers. With H = 8 highways in our dataset, this is effectively O(V) theoretically, but the constant factor from state management grows noticeably at large graph sizes.

What it optimizes: Number of distinct highway transitions along the path. Produces the route that requires the fewest decisions/exits for the driver.

---

3. Implementation

Our implementation is contained in `AlgoTest.java` and supports two operating modes:

Mode 1 — Synthetic graphs (timing experiments): Creates random adjacency-matrix graphs of varying sizes using `createGraph(int size)`, then runs both `dijkstra()` and `bfs()` on the same source-destination pair and records wall-clock time via `System.nanoTime()`.

Mode 2 — TMG file (real highway data): Loads a METAL `.tmg` file using `loadTMG(String filename)`, which parses vertex coordinates, computes edge distances using the Haversine formula, and maps highway labels to integer IDs. Then runs `dijkstraTMG()` and `bfsTMG()` on randomly generated source-destination pairs.

---

4. Data

4.1 Timing Data

We ran both algorithms on synthetic random graphs ranging from **n = 100 to n = 5,000 vertices** in steps of 100, with **5 independent trials per size** (250 total runs per algorithm). Graphs were generated with approximately 40% edge density and 5 distinct highway labels. Results were collected in `timings.txt`.

Format: `graph_size dijkstra_result bfs_result dijkstra_time_sec bfs_time_sec`

4.2 Route Data

We applied both algorithms to **50 randomly selected source-destination pairs** on a real METAL highway graph of the Siena/Albany area region. Results were collected in `results.txt`.

Format: `pair dijkstra_dist_mi bfs_switches`

---

5. Empirical Results

5.1 Runtime Scaling

Graph Size	Dijkstra Avg	BFS Avg
n = 100	0.33 ms	0.28 ms
n = 1,000	3.10 ms	3.48 ms
n = 2,500	18.16 ms	28.67 ms
n = 5,000	76.19 ms	101.30 ms

At small graph sizes (n < 700), BFS is marginally faster due to its lower per-vertex overhead. However, starting around n = 1,000, Dijkstra consistently pulls ahead. At n = 5,000, Dijkstra is 24.8 % faster than BFS.

Across all 50 graph sizes tested, Dijkstra won in 43 out of 50 sizes (86%**)**. The average BFS-to-Dijkstra speed ratio across all sizes was peaking at 1.76 × at n = 4,100.

5.2 Consistency

A striking finding is the difference in variability between the two algorithms. Dijkstra's standard deviation across trials was 0.43 ms on average, extremely stable. BFS had an average standard deviation of 9.43 ms.. This is visible in the BFS timing data, where a single trial at n = 5,000 ranged from 70.23 ms to 162.92 ms across five runs.

This variability is explained by BFS's dependence on graph structure: the deque reordering behavior changes significantly depending on how many zero-cost vs. one-cost transitions exist in a particular random graph.

5.3 Distance vs. Highway Switches (Real Highway Data)

Across 50 real route pairs on the Siena/Albany METAL graph:

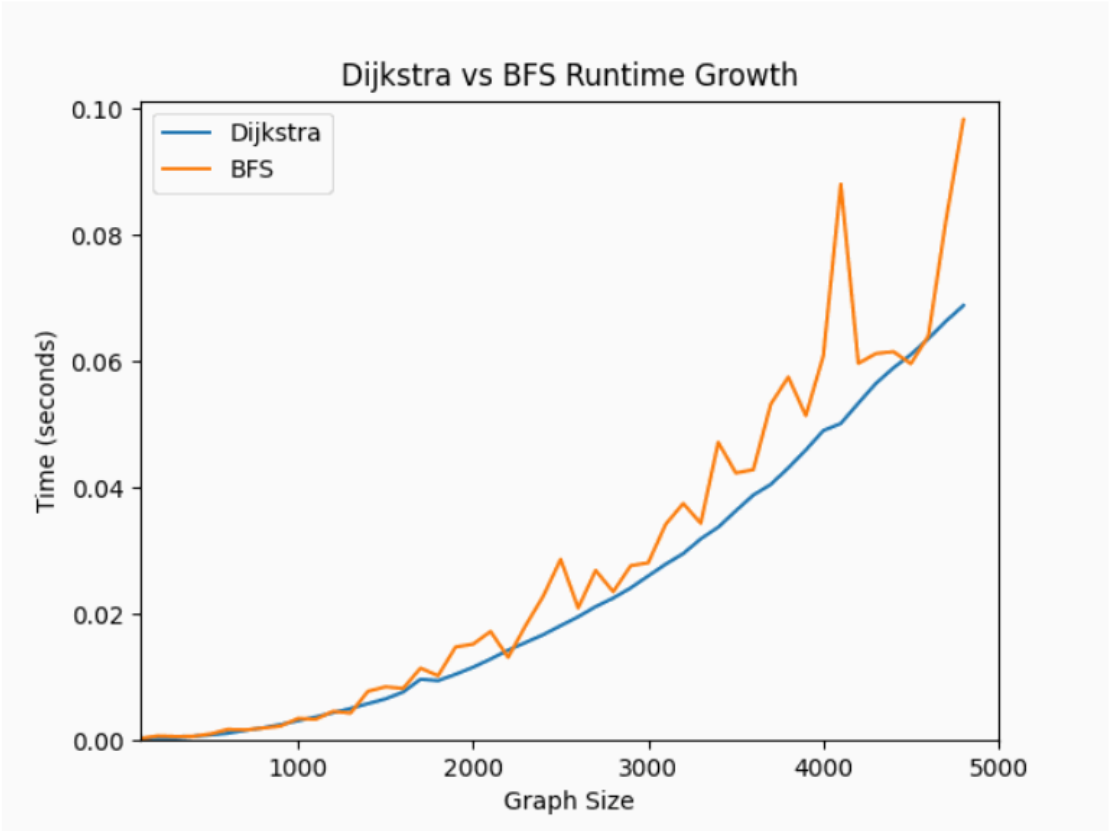
Metric	**Value**
Shortest distance (min)	11.30 mi
Shortest distance (max)	343.41 mi
Average distance	156.52 mi
Min highway switches	1
Max highway switches	13
Average highway switches	7.66

The correlation between Dijkstra distance and BFS switch count is r = 0.65 ms.This means longer routes tend to have more highway switches, but the relationship is far from deterministic. Short trips can require many switches, and long trips can be simple. This confirms that the two algorithms are solving genuinely different problems, and neither result is predictable from the other.

Only 10 out of 50 routes (20%) required 5 or fewer switches. 15 out of 50 routes (30%) required 10 or more switches: underlining that route simplicity is a real and distinct concern from route distance.

---

6. Analysis and Discussion



6.1 Why Dijkstra Is Faster in Practice

Despite BFS having better theoretical complexity on adjacency list graphs ($O(V \times H)$ vs. $O(V^2)$ for our matrix Dijkstra), Dijkstra wins in practice because of implementation-level differences. BFS manages a $V \times H$ state matrix, pushes and pops from both ends of a deque, and must check for stale states on every dequeue operation. These constant factors are significant. On our synthetic graphs, the deque overhead consistently outweighs the theoretical advantage of the lower asymptotic complexity.

This is an important lesson: theoretical complexity does not always predict real-world performance, especially when the constant factors and data structure overhead differ substantially.

6.2 When BFS Has Value

Despite being slower, BFS serves a purpose that Dijkstra cannot fulfill: it finds the route a driver can follow with the fewest highway changes. On a route requiring 10 switches under Dijkstra's distance-optimal path, BFS might find an alternative path that requires only 6 switches at a modest distance cost. Neither result is "wrong", they optimize for different human priorities.

---

7. Metal Implementation

We wanted to create a demo that actually visualizes the difference between Dijkstra and our Modified BFS. Our implementation shows the Priority Queue and a Spanning Tree. We also display the Visited nodes as well as the final Path it took. Here are the different components specifically in the visualization:

Map: A 48-node synthetic graph is procedurally generated using a seeded random number generator. Edges are drawn between nodes based on grid adjacency plus random long-distance connections, mirroring the structure of a real highway graph. Each edge is assigned one of eight highway labels (I-87, US-9, NY-7, etc.) and a weight derived from Euclidean distance. The canvas is rendered using the HTML5 Canvas API and redraws after every algorithm step. Node colors update in real time to reflect their current state:

- **Blue**: undiscovered
- **Amber/orange**: currently in the priority queue or deque
- **Red** :the node being processed this step
- **Green**: confirmed in the spanning tree (shortest path found to this node)
- **White border**: designated start or end node

Sidebar(Buttons):

- The **Dijkstra / BFS buttons:** at the top switch between algorithms and reset state.
- The **Start and End node badges** show the currently selected source and destination; clicking either badge enters selection mode so the viewer can click any node on the map to reassign it.
- The **Start, Step, Play, and Reset buttons** control execution, Step advances one iteration at a time (useful for explaining to an audience), Play runs automatically at the selected speed (1×, 2×, 4×, 8×), and Reset clears all states.
- The **speed selector** controls the playback interval in milliseconds.

Sidebar(Tabs): live data structures Below the pseudocode, four data structure panels update after every step:

- **Priority queue / Deque** — shows every node currently waiting to be processed, labeled with its current best distance (Dijkstra) or switch count (BFS)
- **Spanning tree** — shows every node that has been finalized and added to the shortest-path tree
- **Visited** — shows every node that has been fully processed and will not be revisited
- **Final path** — once the destination is reached, shows the complete node sequence of the optimal route as a chain of labeled badges

8. Conclusions

1. **Dijkstra's algorithm is faster**: winning in 86% of tested graph sizes
2. **Dijkstra is significantly more consistent**: BFS exhibits 21.8 × greater timing variance, making it less reliable for time-critical routing applications
3. **BFS is uniquely useful**: it is the only algorithm we studied that can minimize highway switches, a genuinely different and practically meaningful routing objective
4. **The two algorithms are not interchangeable**: a correlation of $r = 0.65$ between distance and switches means the two objectives diverge frequently enough to matter in practice
5. **Real highways are complex**: 30% of tested routes required 10 or more highway switches on the distance-optimal path, demonstrating that switch minimization is a legitimate concern for drivers

---

9. References

1. Teresco, J. D. METAL: Map-based Educational Tools for Algorithm Learning. Siena College. https://courses.teresco.org/metal/

---

10. AI Assistance Documentation

- Used Claude to build an interactive HTML/JavaScript visualization of the algorithm (similar to METAL HDX) for our showcase demonstration — we provided the algorithm descriptions and Claude produced the canvas-based step-through animation

---

10. Group Member Contributions

Member	**Contributions**
Dilan	Completed the Metal Demo as well as this writeup
Sam Panackal	Contributed to the different algorithms and helped with the creation of the timings.txt
Samiullah Neyazy	Contributed to the different algorithms and helped with the creation of the results.txt

---